

Laboratory 2 — Android + Firebase Team Project

BookNest — Authentication & Firestore CRUD

Student: Ozodbek Tursunaliyev **Student ID:** 57264 **Course:** Cloud Computing Architecture, Akademia WSB **Submission type:** Individual project (solo — all roles performed by the student)

Table of Contents

1. Executive Summary
 2. Definition of Done
 3. Firebase Authentication
 4. Firestore CRUD Operations
 5. Extensions Implemented
 6. Development Workflow
 7. Deliverables Checklist
 8. Retrospective
 9. Conclusion
-

1. Executive Summary

Lab 2 extended the BookNest foundation from Lab 1 from a planned design into a **working application**. Because teammates were absent, the project continued to be delivered individually, with all four roles (Product Owner, Tech Lead, Developer, QA/DevOps) handled by one student, as permitted by the lab guidelines.

By the end of the session BookNest had:

- Working **email/password** authentication (register, log in, log out, session persistence)
- A live **book recommendation list** backed by Cloud Firestore
- Complete **CRUD** (Create, Read, Update, Delete) for the **books** entity
- **Four extensions:** search/filter, sorting, input validation, and a status toggle

Session timeline

Phase	Activity	Duration
1	Planning & choosing extensions	20 min
2	Firebase Authentication	40 min
3	Firestore CRUD	50 min
4	Extensions & polish	30 min
5	Demo & micro-retrospective	40 min

2. Definition of Done

The team (solo) agreed the following "Definition of Done" for Lab 2:

- ☒ User can register and log in with email/password
- ☒ User can log out and return to login screen
- ☒ Main list screen displays data from Firestore in real time
- ☒ User can add a new book recommendation
- ☒ User can edit an existing recommendation (owner only)
- ☒ User can delete a recommendation with a confirmation dialog
- ☒ At least 2 extensions implemented (achieved **4**)

3. Firebase Authentication

3.1 Enabling Email/Password

In the Firebase Console → **Authentication** → **Sign-in method**, the **Email/Password** provider was enabled.

[SCREENSHOT 1] — Firebase Console showing Email/Password provider *Enabled*.

3.2 Login & Register screens

Two Jetpack Compose screens were created (consistent with the Compose choice from Lab 1):

- **Login screen** — email field, password field, *Log in* button, *Register* link, inline error message area, and a loading spinner.
- **Register screen** — email, password, confirm-password, and optional university fields, plus a *Create account* button and *Back to login* link.

[SCREENSHOT 2] — BookNest Login screen running in the emulator. **[SCREENSHOT 3]** — BookNest Register screen.

Source: `LoginScreen.kt` , `RegisterScreen.kt`

3.3 FirebaseAuth implementation

All authentication logic lives in `AuthViewModel.kt` . Key decisions:

- **Registration** calls `createUserWithEmailAndPassword` , then writes a matching document to the Firestore `users` collection (userId, email, displayName, university, createdAt).
- **Login** calls `signInWithEmailAndPassword` .
- **Logout** calls `auth.signOut()` and flips the UI state back to the login screen.
- Calls use Kotlin **coroutines** (`.await()`) inside `viewModelScope` , so the UI thread is never blocked.

```
val result = auth.createUserWithEmailAndPassword(email.trim(), password).await()
val uid = result.user?.uid ?: error("Missing user id after sign-up")
db.collection("users").document(uid).set(user).await()
```

3.4 Navigation rules

Navigation is centralized in `BookNestNavigation.kt` . A `LaunchedEffect` observes the authentication state:

- On app start, if `auth.currentUser != null` , the user **skips the login screen**.
- After successful login/registration the app navigates to the list and calls `popUpTo(0) { inclusive = true }` , which **clears the back stack** so the system Back button cannot return to the login screen (the Compose-Navigation equivalent of the `FLAG_ACTIVITY_CLEAR_TASK` pattern in the troubleshooting guide).

3.5 Error handling

Raw Firebase exceptions are mapped to friendly messages in `friendlyMessage()` :

Firestore error	Shown to user
<code>INVALID_LOGIN_CREDENTIALS</code>	"Incorrect email or password"
<code>email address is already in use</code>	"An account with this email already exists"
<code>network</code> errors	"Network error — check your connection"

Client-side validation (via `Validators`) runs **before** any network call, so malformed emails and short passwords fail instantly with a clear message.

[SCREENSHOT 4] — Login screen showing the "Incorrect email or password" error.

4. Firestore CRUD Operations

4.1 Main entity

The chosen entity is `books` (book recommendations). The data class is `Book.kt`, with default values and a no-arg constructor so Firestore's `toObject<Book>()` works.

4.2 Read (real-time)

`BookViewModel.kt` attaches an `addSnapshotListener` to the `books` collection ordered by `createdAt`. The list updates the instant any document changes — no manual refresh.

[SCREENSHOT 5] — Book list screen showing recommendations from Firestore.

4.3 Create

The *Recommend a book* form (`AddBookScreen.kt`) validates input, then writes a new document. The new book appears in the list immediately through the snapshot listener.

[SCREENSHOT 6] — Add-book form filled in.

4.4 Update

Tapping a book opens `BookDetailScreen.kt`. The recommender sees editable fields; `updateBook()` writes the changes.

4.5 Delete

Owners get a delete action that shows an **AlertDialog confirmation** before `deleteBook()` removes the document.

[SCREENSHOT 7] — Delete confirmation dialog.

4.6 User-specific data & permissions

- Each book stores `recommendedBy` = the creator's `userId` .
- **Only the owner** sees the edit/delete UI (`book.recommendedBy == currentUserId`).
- This is enforced server-side too, in `firestore.rules` : `update, delete` require `resource.data.recommendedBy == request.auth.uid` .

5. Extensions Implemented

Four extensions were completed (the lab required 2–3):

#	Extension	Where
1	Search / filter — case-insensitive across title, author, genre	<code>BookFilters.search</code> , search box on list screen
2	Sorting — Title, Author, Genre (A–Z), Newest first	<code>BookFilters.sort</code> , sort menu in top bar
3	Input validation — required fields, min title length, max 500-char description	<code>Validators.validateBookInput</code>
4	Status toggle — Want-to-read ↔ Finished on saved books	<code>BookViewModel.toggleSavedStatus</code> , <code>SavedBooksScreen</code>

A fifth nicety — an **empty state** message — is shown when the list (or a search) returns nothing.

[SCREENSHOT 8] — Search box filtering the list to one result. **[SCREENSHOT 9]** — Want-to-Read list with a book marked "Finished".

6. Development Workflow

Working solo, commits were made with descriptive messages preserving a clear history. In a team setting each change would go through a feature branch and PR; the recommended branch names are

documented for reference:

- `feature/auth` — authentication screens & FirebaseAuth
- `feature/firestore-crud` — book list + add/edit/delete
- `feature/extensions` — search, sort, validation, status toggle

Trello board: cards for Authentication, CRUD, and each extension were moved across **Backlog** → **Sprint 1** → **In Progress** → **Review** → **Done**.

[SCREENSHOT 10] — Trello board with Sprint 1 cards in Done. [SCREENSHOT 11] — GitHub commit history / pull requests.

7. Deliverables Checklist

Authentication

- ☒ Email/password enabled in Firebase Console
- ☒ Login & Register screens functional
- ☒ Users can create accounts, log in, log out
- ☒ App checks auth status on startup; logged-in users skip login
- ☒ Friendly error messages for invalid credentials

Firestore CRUD

- ☒ `books` collection + Kotlin data class
- ☒ List screen with real-time snapshot listener
- ☒ Add / edit / delete with confirmation
- ☒ List filtered/permissioned to the owner for edits

Extensions

- ☒ 4 extensions implemented (search, sort, validation, status toggle)

Collaboration

- ☒ Trello updated, GitHub has new commits
-

8. Retrospective

What went well - Centralizing auth in a single ViewModel kept navigation logic clean. - The Firestore snapshot listener made the UI feel instant with very little code. - Reusing pure

`Validators` / `BookFilters` functions made extensions quick.

What didn't go well - First Firestore reads failed with *Permission denied* until security rules were switched from locked defaults to "authenticated users". - Compose state hoisting took a couple of attempts to get the search box to recompose correctly.

What I will improve next time - Write the security rules at the same time as the first query. - Extract more logic into pure functions early, to make Lab 3 testing easier.

9. Conclusion

Lab 2 turned BookNest into a functioning MVP: students can register, log in, and manage book recommendations stored in the cloud, with four polish extensions on top. The clean separation between pure logic (`util/`), Firebase access (ViewModels), and Compose UI set up Lab 3 (advanced feature + unit testing) to proceed smoothly.